TikuKits ML: Naive Bayes for Embedded Systems

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

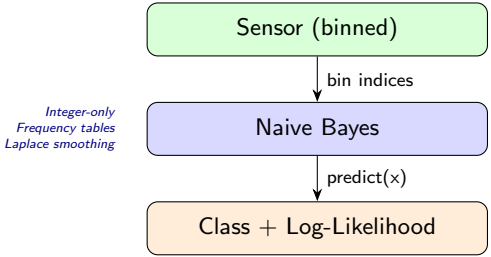
March 14, 2026

Outline

Why Naive Bayes on MCUs?

Embedded Use Cases

- **Categorical sensor data** — binned ADC readings
- **Activity recognition** — motion state classification
- **Fast training** — single-pass frequency counting
- **Multi-class** — handles 2+ classes naturally



Constraints

- No FPU (16-bit MSP430)
- No heap allocation
- Limited RAM (2–8 KB)
- No `exp()`, `log()`, or `sqrt()`

Design Goal

Categorical Naive Bayes using **frequency tables** and **integer $\log_2$** approximation. No floating point, no transcendental functions.

Categorical Naive Bayes

Bayes' theorem:

$$P(c \mid \mathbf{x}) \propto P(c) \prod_{j=1}^{n} P(x_j \mid c)$$

Log-space scoring (avoids products of small numbers):

$$\text{score}(c) = \log_2 P(c) + \sum_{j=1}^{n} \log_2 P(x_j \mid c)$$

With Laplace smoothing:

$$P(x_j = v \mid c) = \frac{\text{freq}[c][j][v] + 1}{\text{count}_c + B}$$

where B = number of bins.

Prediction: $\hat{y} = \arg\max_c \text{score}(c)$

Integer $\log_2$ Approximation

Uses a bit-scan (count leading zeros) to compute $\lfloor \log_2(n) \rfloor$ in Q(shift):

$$\text{log2_q}(n) = \lfloor \log_2(n) \rfloor \times 2^{\text{shift}}$$

No floating point, no lookup tables larger than a word.

Frequency Table

Feature j

bin	b_0	b_1	b_2	b_3
c_0	3	1	0	2
c_1	0	2	4	1

The tiku_kits_ml_nbayes Structure

ml/classification/tiku_kits_ml_nbayes.h

```c
struct tiku_kits_ml_nbayes {
    uint16_t freq
        [TIKU_KITS_ML_NBAYES_MAX_CLASSES]
        [TIKU_KITS_ML_NBAYES_MAX_FEATURES]
        [TIKU_KITS_ML_NBAYES_MAX_BINS];
    uint16_t class_count
        [TIKU_KITS_ML_NBAYES_MAX_CLASSES];
    uint16_t n_total;
    uint8_t  n_features;
    uint8_t  n_bins;
    uint8_t  n_classes;
    uint8_t  shift;
};
```

Memory footprint:

$4 \times 4 \times 16 \times 2 + 4 \times 2 + 2 + 4$

$= 522$ bytes (default config).

Configuration macros

```c
/* Max output classes */
#ifndef TIKU_KITS_ML_NBAYES_MAX_CLASSES
#define TIKU_KITS_ML_NBAYES_MAX_CLASSES 4
#endif

/* Max input features */
#ifndef TIKU_KITS_ML_NBAYES_MAX_FEATURES
#define TIKU_KITS_ML_NBAYES_MAX_FEATURES 4
#endif

/* Max bins per feature */
#ifndef TIKU_KITS_ML_NBAYES_MAX_BINS
#define TIKU_KITS_ML_NBAYES_MAX_BINS 16
#endif

/* Default Q-format shift */
#ifndef TIKU_KITS_ML_NBAYES_SHIFT
#define TIKU_KITS_ML_NBAYES_SHIFT 8
#endif
```

Complete API

Function	Description
`tiku_kits_ml_nbayes_init(nb, nf, nb, nc, sh)`	Initialize with `nf` features, `nb` bins, `nc` classes, Q-format
`tiku_kits_ml_nbayes_reset(nb)`	Clear all frequency tables and counts
`tiku_kits_ml_nbayes_train(nb, x, y)`	Train: increment frequency counts for binned features
`tiku_kits_ml_nbayes_predict(nb, x, &result)`	Return class with highest log-likelihood
`tiku_kits_ml_nbayes_predict_log_proba(nb, x, s)`	Return all class log-likelihood scores in Q(shift)
`tiku_kits_ml_nbayes_count(nb)`	Total number of training samples seen

Input Format

Features must be **pre-binned** to `uint8_t` values in $[0, n_bins)$. The caller is responsible for discretizing continuous sensor values into bin indices before calling `train()` or `predict()`.

Training and Prediction

Training: frequency counting

```c
int tiku_kits_ml_nbayes_train(
    struct tiku_kits_ml_nbayes *nb,
    const uint8_t *x,
    uint8_t y)
{
    uint8_t j;

    /* Increment per-feature bin count */
    for (j = 0; j < nb->n_features; j++) {
        nb->freq[y][j][x[j]]++;
    }

    /* Increment class and total counts */
    nb->class_count[y]++;
    nb->n_total++;

    return TIKU_KITS_ML_OK;
}
```

O(n_features) per training sample.
No matrix operations, no gradient descent.

Prediction: log-likelihood scoring

```c
/* For each class c: */
score = log2_q(class_count[c]);

for (j = 0; j < n_features; j++) {
    /* Laplace-smoothed likelihood */
    score += log2_q(
        freq[c][j][x[j]] + 1);
    score -= log2_q(
        class_count[c] + n_bins);
}

/* Predict: argmax over all scores */
```

Laplace Smoothing

Adding 1 to numerator and B to denominator prevents zero-probability bins from dominating. Critical for sparse data on embedded targets.

Example 1: Temperature Zone Classification

examples/example_ml_nbayes.c — 3 zones, 5 bins

```c
struct tiku_kits_ml_nbayes nb;
uint8_t result;

/* 1 feature, 5 bins, 3 classes, Q8 */
tiku_kits_ml_nbayes_init(&nb, 1, 5, 3, 8);

/* Bin mapping: [0..100)->0, [100..200)->1, [200..300)->2,
 *              [300..400)->3, [400+)->4 */

/* Train: cold=0, normal=1, hot=2 */
uint8_t cold_bins[]   = {0, 0, 1, 0, 1};    /* low bins */
uint8_t normal_bins[] = {2, 2, 2, 1, 2};    /* mid bins */
uint8_t hot_bins[]    = {3, 4, 3, 4, 3};    /* high bins */

uint8_t i;
for (i = 0; i < 5; i++) {
    tiku_kits_ml_nbayes_train(&nb, &cold_bins[i],   0);
    tiku_kits_ml_nbayes_train(&nb, &normal_bins[i], 1);
    tiku_kits_ml_nbayes_train(&nb, &hot_bins[i],    2);
}

/* Classify: bin 2 -> normal */
uint8_t query = 2;
tiku_kits_ml_nbayes_predict(&nb, &query, &result);  /* result = 1 */
```

Example 2: Activity Recognition

2-feature: magnitude + frequency

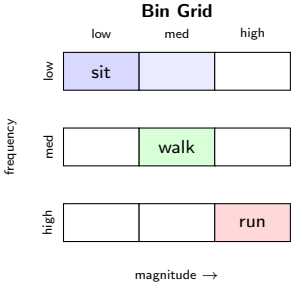
```
struct tiku_kits_ml_nbayes nb;
uint8_t result;
int32_t scores[3];

/* 2 features, 4 bins, 3 classes, Q8 */
tiku_kits_ml_nbayes_init(&nb, 2, 4, 3, 8);

/* sitting=0: low mag, low freq */
uint8_t sit1[] = {0, 0};
uint8_t sit2[] = {0, 1};
tiku_kits_ml_nbayes_train(&nb, sit1, 0);
tiku_kits_ml_nbayes_train(&nb, sit2, 0);

/* walking=1: med mag, med freq */
uint8_t walk1[] = {1, 2};
uint8_t walk2[] = {2, 2};
tiku_kits_ml_nbayes_train(&nb, walk1, 1);
tiku_kits_ml_nbayes_train(&nb, walk2, 1);

/* running=2: high mag, high freq */
uint8_t run1[] = {3, 3};
uint8_t run2[] = {3, 2};
tiku_kits_ml_nbayes_train(&nb, run1, 2);
```



Log-Likelihood Scores

`predict_log_proba()` returns per-class scores in Q(shift). Higher score = more likely class. Useful for confidence thresholding.

Test Cases Overview

#	Test Function	What It Verifies
1	`test_kits_ml_nbayes_init`	Initialization, parameter bounds validation
2	`test_kits_ml_nbayes_simple`	1-feature, 2-class classification
3	`test_kits_ml_nbayes_two_features`	2-feature classification
4	`test_kits_ml_nbayes_three_class`	3-class classification
5	`test_kits_ml_nbayes_log_proba`	Log-likelihood score output
6	`test_kits_ml_nbayes_smoothing`	Laplace smoothing on unseen bins
7	`test_kits_ml_nbayes_imbalanced`	Class imbalance handling (prior effect)
8	`test_kits_ml_nbayes_reset`	Reset clears all frequency tables
9	`test_kits_ml_nbayes_null_inputs`	All functions reject NULL with ERR_NULL

Summary

What We Built

- Categorical Naive Bayes classifier
- Frequency-table based (no matrix ops)
- Integer $\log_2$ scoring (no FPU)
- Laplace smoothing for robustness
- ~522 bytes state (default config)

Key Design Choices

- Pre-binned input (caller discretizes)
- Log-space scoring avoids tiny products
- Bit-scan $\log_2$ replaces `log()`
- Smoothing prevents zero-probability bins

Use Cases

- Temperature zone classification
- Activity recognition (sit/walk/run)
- Categorical sensor pattern matching
- Multi-class embedded decisions

Files

Header:	`ml/classification/tiku_kits_ml_nbayes.h`
Source:	`ml/classification/tiku_kits_ml_nbayes.c`
Tests:	`tests/ml/test_kits_ml_nbayes.c`
Example:	`examples/example_ml_nbayes.c`

Thank You

TikuKits ML: Naive Bayes

Integer-Only Machine Learning for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
Common:	`tikukits/ml/tiku_kits_ml.h`
Header:	`tikukits/ml/classification/tiku_kits_ml_nbayes.h`
Source:	`tikukits/ml/classification/tiku_kits_ml_nbayes.c`
Tests:	`tikukits/tests/ml/test_kits_ml_nbayes.c`
Examples:	`tikukits/examples/example_ml_nbayes.c`